

© Sumit Raj 2019

Sumit Raj, *Building Chatbots with Python*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-4096-0_2

2. Natural Language Processing for Chatbots

Sumit Raj¹

(1) Bangalore, Karnataka, India

This chapter is to get you started with Natural Language Processing (NLP) using Python needed to build chatbots. You will learn the basic methods and techniques of NLP using an awesome open-source library called spaCy . If you are a beginner or intermediate to the Python ecosystem, then do not worry, as you'll get to do every step that is needed to learn NLP for chatbots. This chapter not only teaches you about the methods in NLP but also takes real-life examples and demonstrates them with coding examples. We'll also discuss why a particular NLP method may be needed for chatbots. Note that NLP in itself is a skill.

We'll be closely taking a look at POS tagging, stemming, entity detection, stopwords, dependency parsing, and noun chunks and finding similarity between words. All of these methods will be very helpful for you when you are building chatbots of your use-case.

There are a lot more NLP methods than the ones covered in this chapter. Based on your need for the chatbot you are building, you can try to learn them. The SpaCy library that we are going to learn to use by the end of this chapter will give you enough idea on how to scale your knowledge base of NLP and its understanding. So, let's get started and first try to understand NLP for chatbots in the next section.

Why Do I Need to Know Natural Language Processing to Build a Chatbot?

To understand the answer to this question, let's first understand Natural Language Processing (NLP).

Natural Language Processing (NLP) is a field of Artificial Intelligence that enables computers to analyze and understand the human language.

Now, to perform NLP or, say, Natural Language Understanding (NLU), we have lots of methods that we are going to discuss next. You heard a new term **Natural Language Understanding (NLU)** —what is that now?

In simple terms, NLU is a subset of a bigger picture of NLP, just like machine learning, deep learning, NLP, and data mining are a subset of a bigger picture of Artificial Intelligence(AI), which is an umbrella term for any computer program that does something smart.

A good rule of thumb is to use the term NLU to express a machine's ability to understand the natural language in a form provided by humans.

Now, coming to the question of whether you really need to know NLP to build a chatbot—the answer is both Yes and No. Confused? You heard it right, it's not that you can't build a chatbot at all if you don't know the method and techniques of NLP, but your scope will be somewhat limited. You won't be able to scale the application and keep the code clean at the same time. NLP gives your chatbot the wings to fly when it can't just walk and run.

Chatbots for a common person are nothing but a way to communicate with some intelligent machine at the other end. This machine can be either voice-based or text-based, where the user will provide input in their own language, which in computer science is generally called natural language.

We know that there is no black box that does the magic, and everything works just fine. One should know that there is nothing artificial in AI; it's actually machine learning and deep learning algorithms written by great people, running under the hood. Machines haven't reached a stage where they can think as similar as humans to have their own intelligence. AI systems today—what they do and the way they behave—are the outcome of how we have trained them.

So, to understand the natural language of the user in whatsoever language it may be, or whatever input form it may be (text, voice, image, etc.), we have to write algorithms and use techniques of NLP. NLP is considered the brain of chatbots that processes the raw data, does the munging, cleans it, and then prepares to take appropriate actions.

NLP in itself is a huge topic and requires time and perseverance to learn completely, but there are few methods that are necessary for a chatbot developer to know, which we are going to learn in this chapter.

What Is spaCy?

spaCy is an open-source software library for advanced NLP, written in Python and Cython, built by Matthew Honnibal. It provides intuitive APIs to access its methods trained by deep learning models.

spaCy offers the fastest syntactic parser in the world. Taken directly from spaCy's documentation, they have some amazing benchmarking results, which are shown below.

Benchmarks Results of spaCy

Two peer-reviewed papers in 2015 confirmed that spaCy offers the fastest syntactic parser in the world and that its accuracy is within 1% of the best available. The few systems that are more accurate are 20 times slower or more. Lets try to look at Figure [2-1](#) which shows spaCy benchmarking results based on its speed and accuracy compared to other libraries.

SYSTEM	YEAR	LANGUAGE	ACCURACY	SPEED (WPS)
spaCy v2.x	2017	Python / Cython	92.6	n/a ☹
spaCy v1.x	2015	Python / Cython	91.8	13,963
ClearNLP	2015	Java	91.7	10,271
CoreNLP	2015	Java	89.6	8,602
MATE	2015	Java	92.5	550
Turbo	2015	C++	92.4	349

Figure 2-1 spaCy benchmarking results

spaCy also offers statistical neural network models for a wide range of languages like English, German, Spanish, Portuguese, French, Italian, Dutch, and multi-language NER. It also provides tokenization for various other languages. This table shows speed benchmarked by Choi et al., so it wouldn't be fair to compare spaCy v2.x benchmarked on different hardware. This is the reason you do not see the speed column value for spaCy v2.x.

What Does spaCy Provide?

There are three primary things that spaCy claims to provide and with which it is extremely helpful. Let's try to look at those and understand why one should know and use spaCy as a go-to module for doing NLP.

World's Fastest Library

spaCy does extremely well at extracting large-scale information. It is written from scratch with utmost care for memory with help of the Cython library.

Get Things Done

spaCy is designed with "get things done" in mind. It helps us in accomplishing real-world NLP scenarios. The clean documentation saves a lot of time for developers and computational linguistics enthusiasts and makes them more productive. It's easy to install, just like any other Python package.

Deep Learning

spaCy is one of the best libraries available in the open-source community to process text for deep-learning algorithms. It collaborates seamlessly with TensorFlow, PyTorch, scikit-learn, Gensim, and the rest of Python's related technologies. Deep learning developers can easily construct linguistically sophisticated statistical models for a range of NLP/NLU problems.

Features of spaCy

No other NLP library provides an extremely wide range of APIs to do almost everything, which is what spaCy does. The best things about this library is it is

continuously evolving and getting better and better. Let's have a sneak peek at the features of spaCy as mentioned on their official website [<https://spacy.io/>].

- Non-destructive tokenization
- Named entity recognition
- Support for 28+ languages
- 13 statistical models for 8 languages
- Pre-trained word vectors
- Easy deep-learning integration
- Part-of-speech tagging
- Labeled dependency parsing
- Syntax-driven sentence segmentation
- Built-in visualizers for syntax and NER
- Convenient string-to-hash mapping
- Export to numpy data arrays
- Efficient binary serialization
- Easy model packaging and deployment
- State-of-the-art speed
- Robust, rigorously evaluated accuracy

Now, let's dive into this awesome module for NLP in Python: spaCy.

Installation and Prerequisites

Before we actually dive into spaCy and code snippets, make sure you have Python installed on your OS. If not, refer to [1].

You can use whichever version of Python you are comfortable with. Most of the systems today come pre-installed with default Python version 2.7.x. We'll be using Python 3 in this chapter. So, if you want to use Python 3, please install Python 3 on your operating system by downloading it from <https://www.python.org/downloads/>. If you have Python 2 installed already, you can use that as well; it may or may not need minor code changes.

We will install spaCy via pip [2].

We are going to use a virtual environment [3] and install spaCy into a user directory.

If you are on macOS/OSX/Linux, follow these steps:

```
Step 1:  
python3 -m pip install -U virtualenv
```

```
Step 2:  
virtualenv venv -p /usr/local/bin/python3 #Make sure you use your own OS path for p
```

```
Step 3:  
source venv/bin/activate
```

Step 4:

```
pip3 install -U spacy # We'll be using spaCy version 2.0.11.
```

The final step may take time, so wait patiently.

If you are on Windows ,just change the Step 3 to

```
venv\Scripts\activate
```

Now, we are going to install *Jupyter Notebook* inside our virtual environment, which we activated in the Step 3. Using *Jupyter Notebook* is much easier and more productive than standard Python interpreter. We will be executing all the snippets in Jupyter Notebook in the coming chapters.

To install Jupyter Notebook, run the following pip command:

```
pip3 install jupyter
```

This command will install Jupyter Notebook in your system.

At this point you should have spaCy and Jupyter Notebook installed in your virtualenv. Let's verify if everything was successfully installed.

1. Go to your command line interface and type the following and you should see a server being started and opening a url in your default browser.

```
$ jupyter notebook
```

The default url is `http://localhost:8888/tree`. It should look something like Figure [2-2](#).

2. Click on New as shown in Figure [2-2](#), and choose Python 3. It will open a new tab in your current browser and create a new notebook for you, where you can play with the Python code. You can execute any Python code, import libraries, plot charts, and markdown cells.

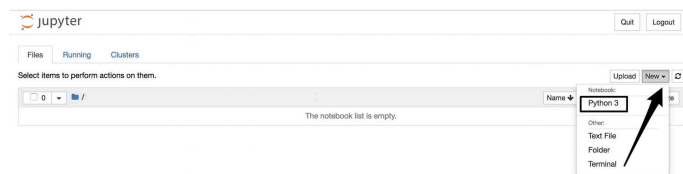


Figure 2-2 Jupyter Notebook first look

3. Type `import spaCy` and run the cell by clicking on "Run" button or by pressing Shift + Enter. It should look something like Figure [2-3](#).

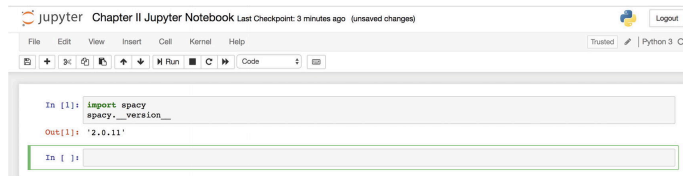


Figure 2-3 Verifying spaCy installation

If Step 3 doesn't throw any error messages, then you have successfully installed the spaCy module on your system. You should see your installed spaCy version in your notebook. If you want to install the same version of spaCy then you can specify the version while installing spaCy via pip.

```
pip3 install -U spacy==2.0.11
```

What Are SpaCy Models?

SpaCy models are just like any other machine learning or deep learning models. A model is a yield of an algorithm or, say, an object that is created after training data using a machine learning algorithm. spaCy has lots of such models that can be placed directly in our program by downloading it just like any other Python package.

Now, we are going to install spaCy models as Python packages.

To do that, we'll run the following command in the notebook by taking use of notebook's magic command. By prefixing! [Exclamation Operator] before the shell command, we can run shell commands as well from Jupyter Notebooks. Let's see how it looks.

```
!python3 -m spacy download en
```

You might get a permission issue while using Jupyter Notebook to download spaCy models for Python 3. Go to your terminal and run the following command:

```
sudo python3 -m download en
```

See Figure [2-4](#) for reference.

```
Sumit:Chapter II geospark-device-3$ sudo python3 -m spacy download en
Password:
Collecting https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-2.0.0/en_core_web_sm-2.0.0.tar.gz
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-2.0.0/en_core_web_sm-2.0.0.tar.gz (37.4MB)
    100% |#####| 37.4MB 59.0MB/s
Requirement already satisfied (use --upgrade to upgrade): en-core-web-sm==2.0.0 from https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-2.0.0/en_core_web_sm-2.0.0.tar.gz in /usr/local/lib/python3.6/site-packages
You are using pip version 10.0.1, however version 18.0 is available.
You should consider upgrading via the 'pip install --upgrade pip' command.

Linking successful
/usr/local/lib/python3.6/site-packages/en_core_web_sm -->
/usr/local/lib/python3.6/site-packages/spacy/data/en

You can now load the model via spacy.load('en')
```

Figure 2-4 Downloading spaCy models

As you can see in Figure [2-4](#), spaCy tries to download some core files and installs them as Python packages.

Note! [Exclamation Operator] works only in Jupyter Notebook. To install spaCy models directly from the terminal, you need to remove! [Exclamation Operator]; otherwise it will result into an error.

- [1] <https://www.python.org/downloads/>
- [2] <https://packaging.python.org/tutorials/installing-packages/#installing-from-pypi>
- [3] <http://docs.python-guide.org/en/latest/dev/virtualenvs/>

Fundamental Methods of NLP for Building Chatbots

It's really important to be good at basics to be an expert at something and do it effectively and efficiently. To build chatbots, we need to know the fundamental methods of NLP. These methods help us break the input into chunks and make sense of it. In the next section, we are going to learn some of the most used NLP methods that will not just help you be good at NLP but also good at building cool chatbots. The more we can process the input text better and more efficiently, the better we can respond to the user.

POS Tagging

Part-of-speech (POS) tagging is a process where you read some text and assign parts of speech to each word or token, such as noun, verb, adjective, etc.

POS tagging becomes extremely important when you want to identify some entity in a given sentence. The first step is to do POS tagging and see what our text contains.

Let's get our hands dirty with some of the examples of real POS tagging.

Example 1:

```
nlp = spacy.load('en') #Loads the spacy en model into a python object
doc = nlp(u'I am learning how to build chatbots') #Creates a doc object
for token in doc:
    print(token.text, token.pos_) #prints the text and POS
```

Output:

```
('I', 'PRON')
('am', 'VERB')
('learning', 'VERB')
('how', 'ADV')
('to', 'PART')
('build', 'VERB')
('chatbots', 'NOUN')
```

Example 2:

```
doc = nlp(u'I am going to London next week for a meeting.')
for token in doc:
    print(token.text, token.pos_)
```

Output:

```
('I', 'PRON')
('am', 'VERB')
('going', 'VERB')
('to', 'ADP')
('London', 'PROPN')
('next', 'ADJ')
('week', 'NOUN')
('for', 'ADP')
('a', 'DET')
('meeting', 'NOUN')
('.', 'PUNCT')
```

As we can see, when we print the tokens from the returned `Doc` object from the method `nlp`, which is a container for accessing the annotations, we get the POS tagged with each of the words in the sentence.

These tags are the properties belonging to the word that determine the word is used in a grammatically correct sentence. We can use these tags as the word features in information filtering, etc.

Let's try to take another example where we try to explore different attributes of the token coming from `Doc` object.

Example 3:

```
doc = nlp(u'Google release "Move Mirror" AI experiment that matches your pose from 80,000 images')
for token in doc:
    print(token.text, token.lemma_, token.pos_, token.tag_, token.dep_,
          token.shape_, token.is_alpha, token.is_stop)
```

Output:

Text	Lemma	POS	Tag	Dep	Shape	Alpha	Stop
Google	google	PROPN	NNP	compound	Xxxxx	True	False
Release	release	NOUN	NN	nmod	xxxx	True	False
"	"	PUNCT	` `	punct	"	False	False
Move	move	PROPN	NNP	nmod	Xxxx	True	False
Mirror	mirror	PROPN	NNP	nmod	Xxxxx	True	False
"	"	PUNCT	“	punct	"	False	False

Text	Lemma	POS	Tag	Dep	Shape	Alpha	Stop
AI	ai	PROPN	NNP	compound	XX	True	False
Experiment	experiment	NOUN	NN	ROOT	xxxx	True	False
That	that	ADJ	WDT	nsubj	xxxx	True	True
Matches	match	VERB	VBZ	relcl	xxxx	True	False
Your	-PRON-	ADJ	PRP\$	poss	xxxx	True	True
Pose	pose	NOUN	NN	dobj	xxxx	True	False
From	from	ADP	IN	prep	xxxx	True	True
80,000	80,000	NUM	CD	num-mod	dd,ddd	False	False
Images	image	NOUN	NNS	pobj	xxxx	True	False

Example 4:

```
doc = nlp(u'I am learning how to build chatbots')
for token in doc:
    print(token.text, token.lemma_, token.pos_, token.tag_, token.dep_,
          token.shape_, token.is_alpha, token.is_stop)
```

Output :

TEXT	LEMMA	POS	TAG	DEP	SHAPE	ALPHA	STOP
I	-PRON-	PRON	PRP	nsubj	X	True	False
am	be	VERB	VBP	aux	xx	True	True
learn- ing	learn	VERB	VBG	ROOT	xxxx	True	False
how	how	ADV	WRB	adv-mod	xxx	True	True
to	to	PART	TO	aux	xx	True	True
build	build	VERB	VB	xcomp	xxxx	True	False
chat- bots	chatbot	NOUN	NNS	dobj	xxxx	True	False

Refer to the below table to find out the meaning of each attribute we printed in the code.

TEXT	Actual text or word being processed
LEMMA	Root form of the word being processed
POS	Part-of-speech of the word
TAG	They express the part-of-speech (e.g., VERB) and some amount of morphological information (e.g., that the verb is past tense).
DEP	Syntactic dependency (i.e., the relation between tokens)
SHAPE	Shape of the word (e.g., the capitalization, punctuation, digits format)
ALPHA	Is the token an alpha character?
Stop	Is the word a stop word or part of a stop list?

You can refer the below table to understand what each POS attribute values of the token object mean. This list gives a detailed idea of part-of-speech tags assigned by spaCy’s models.

POS	DESCRIPTION	EXAMPLES
ADJ	adjective	<i>big, old, green, incomprehensible, first</i>
ADP	adposition	<i>in, to, during</i>
ADV	adverb	<i>very, tomorrow, down, where, there</i>
AUX	auxiliary	<i>is, has (done), will (do), should (do)</i>
CONJ	conjunction	<i>and, or, but</i>
CCONJ	coordinating conjunction	<i>and, or, but</i>
DET	determiner	<i>a, an, the</i>
INTJ	interjection	<i>psst, ouch, bravo, hello</i>
NOUN	noun	<i>girl, cat, tree, air, beauty</i>
NUM	numeral	<i>1, 2017, one, seventy-seven, IV, MMXIV</i>
PART	particle	<i>‘s, not,</i>

POS	DESCRIPTION	EXAMPLES
PRON	pronoun	<i>I, you, he, she, myself, themselves, somebody</i>
PROPN	proper noun	<i>Mary, John, London, NATO, HBO</i>
PUNCT	punctuation	<i>., (,), ?</i>
SCONJ	subordinating conjunction	<i>if, while, that</i>
SYM	symbol	<i>\$, %, \$, ©, +, -, ×, ÷, =, :, ½</i>
VERB	verb	<i>run, runs, running, eat, ate, eating</i>
X	other	<i>sfpkdpsxmsa</i>

SPACE space

So why is POS tagging needed for chatbots?

Answer: to reduce the complexity of understanding a text that can't be trained or is trained with less confidence. By use of POS tagging, we can identify parts of the text input and do string matching only for those parts. For example, if you were to find if a location exists in a sentence, then POS tagging would tag the location word as NOUN, so you can take all the NOUNs from the tagged list and see if it's one of the locations from your preset list or not.

Stemming and Lemmatization

Stemming is the process of reducing inflected words to their word stem, base form.

A stemming algorithm reduces the words "saying" to the root word "say," whereas "presumably" becomes presum. As you can see, this may or may not always be 100% correct.

Lemmatization is closely related to **stemming**, but lemmatization is the algorithmic process of determining the lemma of a word based on its intended meaning.

For example, in English, the verb "to walk" may appear as "walk," "walked," "walks," or "walking." The base form, "walk," that one might look up in a dictionary, is called the *lemma* for the word. spaCy doesn't have any in-built stemmer, as lemmatization is considered more correct and productive.

Difference between Stemming and Lemmatization

- **Stemming** does the job in a crude, heuristic way that chops off the ends of words, assuming that the remaining word is what we are actually looking for, but it often includes the removal of derivational affixes.
- **Lemmatization** tries to do the job more elegantly with the use of a vocabulary and morphological analysis of words. It tries its best to remove inflectional endings only and return the dictionary form of a word, known as the lemma.

Though few libraries provide methods for stemming as well as lemmatization, it's always a best practice to use lemmatization to get the root word correctly.

Let's try to explore lemmatization by taking some of the examples:

Example 1:

```
from spacy.lemmatizer import Lemmatizer
from spacy.lang.en import LEMMA_INDEX, LEMMA_EXC, LEMMA_RULES
lemmatizer = Lemmatizer(LEMMA_INDEX, LEMMA_EXC, LEMMA_RULES)
lemmatizer('chuckles', 'NOUN') # 2nd param is token's part-of-speech tag
```

Output:

```
[u'chuckle']
```

Example 2:

```
lemmatizer('blazing', 'VERB')
```

Output:

```
[u'blaze']
```

Example 3:

```
lemmatizer('fastest', 'ADJ')
```

Output:

```
[u'fast']
```

If you want to see the comparison between a stemmer and lemmatizer, then you need to install one of the most popular libraries for Python: Natural Language Toolkit (NLTK). spaCy has gained popularity very recently, but it was NLTK that made every NLP enthusiast take a plunge into the ocean of NLP and its techniques.

Check the following example where we try to use two stemming techniques provided by NLTK. First, we try to get the stem of the word “fastest” using PorterStemmer and then using SnowBallStemmer. Both of them give the same result—that is, “fastest”—but when we do the lemmatization using spaCy's

method, it gives us “fast” as the stem of “fastest,” which is more meaningful and correct.

```
from nltk.stem.porter import *
from nltk.stem.snowball import SnowballStemmer
porter_stemmer = PorterStemmer()
snowball_stemmer = SnowballStemmer("english")
print(porter_stemmer.stem("fastest"))
print(snowball_stemmer.stem("fastest"))
fastest
fastest
```

Note Make sure to install `nltk` package using `pip3` before trying to run this code.

Since you are pretty much aware what a stemming or lemmatization does in NLP, you should be able to understand that whenever you come across a situation where you need the root form of the word, you need to do lemmatization there. For example, it is often used in building search engines. You must have wondered how Google gives you the articles in search results that you meant to get even when the search text was not properly formulated.

This is where one makes use of lemmatization. Imagine you search with the text, “*When will the next season of Game of Thrones be releasing?*”

Now, suppose the search engine does simple document word frequency matching to give you search results. In this case, the aforementioned query probably won’t match an article with a caption “*Game of Thrones next season release date.*”

If we do the lemmatization of the original question before going to match the input with the documents, then we may get better results.

We’ll try to test this theory as well in upcoming sections.

Named-Entity Recognition

Named-entity recognition (NER), also known by other names like **entity identification** or **entity extraction**, is a process of finding and classifying **named entities** existing in the given text into pre-defined categories.

The NER task is hugely dependent on the knowledge base used to train the NE extraction algorithm, so it may or may not work depending upon the provided dataset it was trained on.

spaCy comes with a very fast entity recognition model that is capable of identifying entity phrases from a given document. Entities can be of different types, such as person, location, organization, dates, numerals, etc. These entities can be accessed through `.ents` property of the `doc` object.

Let’s try to find named-entities by taking some examples with the help of spaCy’s powerful NER tagging capability.

Example 1:

```
my_string = u"Google has its headquarters in Mountain View, California having revenue amounted to 10  
doc = nlp(my_string)  
for ent in doc.ents:  
    print(ent.text, ent.label_)
```

Output:

```
('Google', 'ORG')  
( 'Mountain View', 'GPE')  
( 'California', 'GPE')  
( '109.65 billion US dollars', 'MONEY')
```

We can see how beautifully and automagically the spaCy model could easily identify that the word **Google** as an **Organization**, **California** is a **Geopolitical entity**, and in the given sentence we are talking about **109.65 billion US dollars**, which is actually about money.

Let's try to explore some more examples.

Example 2:

```
my_string = u"Mark Zuckerberg born May 14, 1984 in New York is an American technology entrepreneur a  
doc = nlp(my_string)  
for ent in doc.ents:  
    print(ent.text, ent.label_)
```

Output:

```
('Mark Zuckerberg', 'PERSON')  
( 'May 14, 1984', 'DATE')  
( 'New York', 'GPE')  
( 'American', 'NORP')  
( 'Facebook', 'ORG')
```

Example 3:

```
my_string = u"I usually wake up at 9:00 AM. 90% of my daytime goes in learning new things."  
doc = nlp(my_string)  
for ent in doc.ents:  
    print(ent.text, ent.label_)
```

Output:

```
('9:00 AM', 'TIME')  
( '90%', 'PERCENT')
```

As you can see, the entity extractor can easily extract the time information from the given string. Also as you can see entity extractor not just tries to identify the number but also exact PERCENTAGE value.

As per the documentation by spaCy, models trained on the [OntoNotes 5¹](#) corpus support the following entity types.

TYPE	DESCRIPTION
PERSON	People, including fictional
NORP	Nationalities or religious or political groups
FAC	Buildings, airports, highways, bridges, etc.
ORG	Companies, agencies, institutions, etc.
GPE	Countries, cities, states
LOC	Non-GPE locations, mountain ranges, bodies of water
PRODUCT	Objects, vehicles, foods, etc. (not services)
EVENT	Named hurricanes, battles, wars, sports events, etc.
WORK_OF_ART	Titles of books, songs, etc.
LAW	Named documents made into laws
LANGUAGE	Any named language
DATE	Absolute or relative dates or periods
TIME	Times smaller than a day
PERCENT	Percentage, including “%”
MONEY	Monetary values, including unit
QUANTITY	Measurements, as of weight or distance
ORDINAL	“first,” “second,” etc.
CARDINAL	Numerals that do not fall under another type

Whenever we intend to build a conversational agent or chatbot in simple terms, we always have a domain in mind. For example, we want the chatbot to book a doctor’s appointment, order food, pay a bill, fill out a banking application, e-Commerce, etc. The chatbot could be solving a combination of these problems as well. By finding out the entity in the question, one can get a fair idea of the context in which the question was asked.

Let’s try to understand this by taking an example of two sentences with similar words and different meanings.

```
my_string1 = u"Imagine Dragons are the best band."  
my_string2 = u"Imagine dragons come and take over the city."  
doc1 = nlp(my_string1)
```

```
doc2 = nlp(my_string2)
for ent in doc1.ents:
    print(ent.text, ent.label_)
```

The above `for` loop over `doc1` object gives an output:

```
('Imagine Dragons', 'ORG')
```

Awesome, isn't it? It'll become more intriguing when you will come to realize that the entity recognizer doesn't recognize any entity in 2nd string. Run the following code and `doc2` doesn't produce any output.

```
for ent in doc2.ents:
    print(ent.text, ent.label_)
```

Now, imagine you were to extract the context of the above two strings in a live environment. What would you do? With help of *Entity Extractor*, one can easily figure out the context of the statement and intelligently take the conversation further.

Stop Words

Stop words are high-frequency words like *a, an, the, to* and *also* that we sometimes want to filter out of a document before further processing. Stop words usually have little lexical content and do not hold much of a meaning.

Below is a list of 25 semantically non-selective stop words that are common in Reuters-RCV1.

```
a    an    and    are    as    at    be    by    for
from  has    he    in    is    it    its   of    on
that  the    to    was    were  will  with
```

Let's get into some code and try to understand how things work.

To see all the words defined as stop words in spaCy you can run the following lines of code:

```
from spacy.lang.en.stop_words import STOP_WORDS
print(STOP_WORDS)
```

You should see something like:

```
set(['all', 'six', 'just', 'less', 'being', 'indeed', 'over', 'move', 'anyway', 'fifty', 'four', 'no',
.....
'whereby', 'third', 'i', 'whole', 'noone', 'sometimes', 'well', 'together', 'yours', 'their', 'rathe
```

There are about 305 stop words defined in spaCy's stop words list. You can always define your own stop words if needed and override the existing list.

To see if a word is a stop word or not, you can use the `nlp` object of spaCy.

We can use the `nlp` object's `is_stop` attribute.

Example 1:

```
nlp.vocab[u'is'].is_stop
```

Output:

```
True
```

Example 2:

```
nlp.vocab[u'hello'].is_stop
```

Output:

```
False
```

Example 3:

```
nlp.vocab[u'with'].is_stop
```

Output:

```
True
```

Stop words are a very important part of text clean up. It helps removal of meaningless data before we try to do actual processing to make sense of the text.

Suppose you are in a situation where you are building a bot to make people happy by assessing their mood. Now, one needs to analyze the sentiment in the text input by the user so that correct response can be formulated. Here, before begging to do basic sentiment analysis, we should remove the noise in the data that exists in the form of stop words.

Dependency Parsing

Dependency parsing is one of the more beautiful and powerful features of spaCy that is fast and accurate. The parser can also be used for sentence boundary detection and lets you iterate over base noun phrases, or “chunks.”

This feature of spaCy gives you a parsed tree that explains the parent-child relationship between the words or phrases and is independent of the order in which words occur.

Let's take an example where you have to parse the following sentence:

Book me a flight from Bangalore to Goa

Example 1:

```
doc = nlp(u'Book me a flight from Bangalore to Goa')
blr, goa = doc[5], doc[7]
list(blr.ancestors)
```

Output:

```
[from, flight, Book]
```

The above output can tell us that user is looking to book the flight from Bangalore.

Let's try to list the ancestors of `goa.ancestors` object:

```
list(goa.ancestors)
```

Output:

```
[to, flight, Book]
```

This output can tell us that the user is looking to book the flight to Goa.

What Are Ancestors in Dependency Parsing?

Ancestors are the rightmost token of this token's syntactic descendants. Like in the above example for the object `blr` the ancestors were *from*, *flight*, and *Book*.

Remember you can always list the ancestors of a `doc` object item using `ancestors` attribute.

```
list(doc[4].ancestors) #doc[4]==flight
```

The above code will output:

```
[flight, Book]
```

To check if a `doc` object item is an ancestor of another `doc` object item programmatically, we can do the following:

```
doc[3].is_ancestor(doc[5])
```

The above returns `True` because `doc[3]` (i.e., *flight*) is an ancestor of `doc[5]` (i.e., *Bangalore*). You can try more examples like this to get a better understanding of dependency parsing and the ancestors concept.

If we try to think of a real-world scenario that we might actually face while trying to build a chatbot, we may come across some sentence like

I want to book a cab to the hotel and a table at a restaurant.

In this sentence, it's important to know what *tasks* are requested and where they are targeted (i.e., *whether the user wants to book a cab to the hotel or the restaurant*).

Let's try to do that using the following code:

Example 1:

```
doc = nlp(u'Book a table at the restaurant and the taxi to the hotel')
tasks = doc[2], doc[8] #(table, taxi)
tasks_target = doc[5], doc[11] #(restaurant, hotel)
for task in tasks_target:
    for tok in task.ancestors:
        if tok in tasks:
            print("Booking of {} belongs to {}".format(tok, task))
        break
```

Output:

```
Booking of table belongs to restaurant
Booking of taxi belongs to hotel
```

What Are Children in Dependency Parsing?

Children are immediate syntactic dependents of the token. We can see the children of a word by using `children` attribute just like we used `ancestors`.

```
list(doc[3].children)
```

will output

```
[a, from, to]
```

Interactive Visualization for Dependency Parsing

It's very difficult to understand the complete dependency parsing concept for the first time. spaCy gives an extremely easy and interactive way to understand its dependency parsing. spaCy v2.0+ has a visualization module where we can pass a `Doc` or a list of `Doc` objects to `displaCy` and call `serve` method of `displaCy` to run the web server.

Figure 2-5 shows how the interactive visualization will look for dependency parsing.

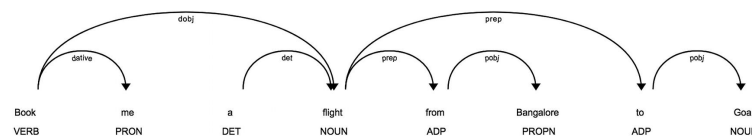


Figure 2-5 Interactive visualization for dependency parsing

You can also generate the dependency parsing visualization in Figure 2-5. To create a visualization like this, run the following code and then go to <http://localhost:5000> in your browser.

Let's try to do the visualization of our example of tasks and the target of the tasks.

```
from spacy import displacy
doc = nlp(u'Book a table at the restaurant and the taxi to the hotel')
displacy.serve(doc, style="dep")
```

Running this code will have output like that in Figure 2-6. If you get something similar, then go to another tab of your browser and enter <http://localhost:5000>.

```
In [*]: from spacy import displacy
doc = nlp(u'Book a table at the restaurant and the taxi to the hotel')
displacy.serve(doc, style="dep")

Serving on port 5000...
Using the 'dep' visualizer
```

Figure 2-6 Starting dependency parsing server on localhost

We get a visualization of dependency parsing for this string in the code (shown in Figure 2-7).

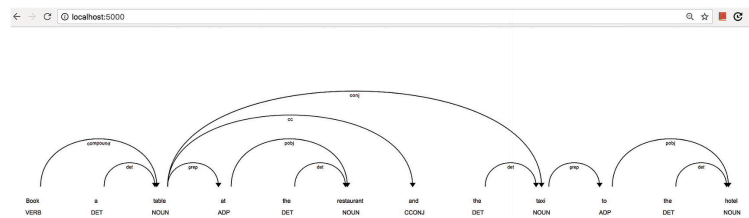


Figure 2-7 Example of dependency parsing

Let's take one more example of dependency parsing where we assume a user is asking the following sentence:

What are some places to visit in Berlin and stay in Lubeck?

We will first create the `doc` object as shown here:

```
doc = nlp(u"What are some places to visit in Berlin and stay in Lubeck")
```

Now, we get the places being talked about and the actions the user wants:

```
places = [doc[7], doc[11]] #[Berlin, Lubeck]
actions = [doc[5], doc[9]] #[visit, stay]
```

Since you already know POS tagging and entity extraction, you can easily get the places and actions automatically.

Now that we have the places, let's iterate through each of its ancestors and check if any ancestors are found in actions. The first parent of place found in the list of actions should be the action for the place in question.

```
for place in places:
    for tok in place.ancestors:
        if tok in actions:
            print("User is referring {} to {}".format(place, tok))
            break
```

Output:

```
User is referring: Berlin to visit
User is referring: Lubeck to stay
```

As we see in these examples, dependency parsing makes it quite easy to understand what the user is referring to. We saw that in the case of two different tasks as well, we can pretty figure out the expectation and, based on that, formulate the next response.

What Is the Use of Dependency Parsing in Chatbots?

Dependency parsing is one the most important parts when building chatbots from scratch. It becomes far more important when you want to figure out the meaning of a text input from your user to your chatbot. There can be cases when you haven't trained your chatbots, but still you don't want to lose your customer or reply like a dumb machine. In these cases, dependency parsing really helps to find the relation and explain a bit more about what the user may be asking for.

If we were to list things for which dependency parsing helps, some might be:

- It helps in finding relationships between words of grammatically correct sentences.
- It can be used for sentence boundary detection.
- It is quite useful to find out if the user is talking about more than one context simultaneously.

You must be wondering, what if your bot user says any grammatically incorrect sentence or uses any SMS textspeak while giving input about something? As discussed in Chapter 1, you have to be cautious about these situations and handle them accordingly using NLP techniques.

You need to write your own custom NLP to understand the context of the user or your chatbot and, based on that, identify the possible grammatical mistakes a user can make.

All in all, you must be ready for such scenarios where a user will input garbage values or grammatically incorrect sentences. You can't handle all such scenarios at once, but you can keep improving your chatbot by adding custom NLP code or by limiting user input by design.

Noun Chunks

Noun chunks or NP-chunking are basically "base noun phrases." We can say they are flat phrases that have a noun as their head. You can think of noun chunks as a noun with the words describing the noun.

Let's try to take an example and understand it better.

Example 1:

```
doc = nlp(u"Boston Dynamics is gearing up to produce thousands of robot dogs")
list(doc.noun_chunks)
```

Output:

```
[Boston Dynamics, thousands, robot dogs]
```

Though having noun chunks from a given sentence helps a lot, spaCy provides other attributes that can be helpful too. Let's try to explore some of those.

Example 2:

```
doc = nlp(u"Deep learning cracks the code of messenger RNAs and protein-coding potential")
for chunk in doc.noun_chunks:
    print(chunk.text, chunk.root.text, chunk.root.dep_,
          chunk.root.head.text)
```

Output:

TEXT	ROOT.TEXT	ROOT.DEP_	ROOT.HEAD.TEXT
deep learning	learning	nsubj	cracks
the code	code	dobj	cracks
messenger RNAs	RNAs	pobj	of
protein-coding potential	potential	conj	RNAs

As we can see from this table we get the noun chunks and their attributes. The following table will help you understand each column.

Column	Meaning
Text	Text of the original noun chunk
Root text	Text of the original word that connects the noun chunk with remaining parse
Root dep	Dependency relation that connects the root to its head
Root head text	Text of the root token's head

Finding Similarity

Finding similarity between two words is a use-case you will find most of the time working with NLP. Sometimes it becomes fairly important to find if two words are similar. While building chatbots you will often come to situations where you don't have to just find similar-looking words but also how closely related two words are logically.

spaCy uses high-quality word vectors to find similarity between two words using *GloVe algorithm* (Global Vectors for Word Representation).

GloVe is an unsupervised learning algorithm for obtaining vector representations for words. GloVe algorithm uses aggregated global word-word co-occurrence statistics from a corpus to train the model.

Let's try to look at the actual values inside the vectors of spaCy using *vector* attribute of the token.

```
doc = nlp(u'How are you doing today?')
for token in doc:
    print(token.text, token.vector[:5])
```

Output:

```
(u'How', array([-0.29742685,  0.73939574, -0.04001453,  0.44034013,  2.8967502 ],      dtype=float32
```

Seeing this output, it doesn't make much sense and meaning. From an application's perspective, what matters the most is how similar the vectors of different words are—that is, the word's meaning itself.

In order to find similarity between two words in spaCy, we can do the following.

Example 1:

```
hello_doc = nlp(u"hello")
hi_doc = nlp(u"hi")
hella_doc = nlp(u"hella")
print(hello_doc.similarity(hi_doc))
print(hello_doc.similarity(hella_doc))
```

Output:

```
0.7879069442766685
0.4193425861242359
```

If you see the word *hello*, it is more related and similar to the word *hi*, even though there is only a difference of a character between the words *hello* and *hella*.

Let's take one more example of a sentence and learn how spaCy does the similarity comparison. Remember our Game of Thrones example in previous sections? We are going to try that and see the similarity.

Code:

```
GoT_str1 = nlp(u"When will next season of Game of Thrones be releasing?")
GoT_str2 = nlp(u"Game of Thrones next season release date?")
GoT_str1.similarity(GoT_str2)
```

Output:

```
0.785019122782813
```

As we can see in this example, the similarity between both of the sentences is about 79%, which is good enough to say that both of the sentences are quite similar, which is true. This can help us save a lot of time for writing custom code when building chatbots. So, we come to a fact that spaCy gives us a meaningful similarity between two words using the word vectors rather than just looking at their spelling or alphabets.

We will take a very simple example and try to find the similarity between words.

```
example_doc = nlp(u"car truck google")
for t1 in example_doc:
    for t2 in example_doc:
        similarity_perc = int(t1.similarity(t2) * 100)
        print "Word {} is {}% similar to word {}".format(t1.text, similarity_perc, t2.text)
```

Output:

```
Word car is 100% similar to word car
Word car is 71% similar to word truck
Word car is 24% similar to word google
Word truck is 71% similar to word car
Word truck is 100% similar to word truck
Word truck is 36% similar to word google
Word google is 24% similar to word car
Word google is 36% similar to word truck
Word google is 100% similar to word google
```

Finding similarity between words or sentences becomes quite important when we intend to build any application that is hugely dependent on the implementations of NLP. If you have ever used StackOverflow, whenever we try to ask a new question, it tries to list similar questions already asked on the platform. This is one of the best examples where finding similarity between two sets of sentences might help. spaCy's confidence to find the similarity between two words based on an already trained model purely depends on the kind of general assumption taken.

When building chatbots, finding similarity can be very handy for the following situations:

- When building chatbots for recommendation
- Removing duplicates
- Building a spell-checker

These things that we learned are really important while building chatbots so that we know how to parse user inputs so that they make sense while

writing business logic inside code.

Good to Know Things in NLP for Chatbots

In this section we are going to learn about a couple of interesting topics that come in handy often when you plan to write your own custom NLP methods to handle certain scenarios. Make sure you go through them, as when you expect it the least, it's needed the most. We will briefly talk about tokenization and usage of regular expressions in a chatbot scenario.

Tokenization

Tokenization is one of the simple yet basic concepts of NLP where we split a text into meaningful segments. spaCy first tokenizes the text (i.e., segments it into words and then punctuation and other things). A question might come to your mind: Why can't I just use the built-in *split* method of Python language and do the tokenization? Python's *split* method is just a raw method to split the sentence into tokens given a separator. It doesn't take any meaning into account, whereas tokenization tries to preserve the meaning as well.

Let's try some code and see how tokenization works.

Example 1:

```
doc = nlp(u'Brexit is the impending withdrawal of the U.K. from the European Union.')
for token in doc:
    print(token.text)
```

Output:

```
Brexit
is
the
impending
withdrawal
of
the
U.K.
from
the
EuropeanUnion
```

If you see in the above output, U.K. comes as a single word after the tokenization process, which makes sense, as U.K. is a country name and splitting it would be wrong. Even after this if you not happy with spaCy's tokenization, then you can use its `add_special_case` case method to add your own rule before relying completely on spaCy's tokenization method.

Regular Expressions

You must already know about regular expressions and their usage. This book assumes you must be familiar with regular expressions in general.

In this section, we are just going to run through some of the examples and see how regular expressions can be beneficial and useful while building chatbots.

Text analysis and processing is a big subject in itself. Sometimes words play together in a way that makes it extremely difficult for machines to understand and get trained upon.

Regular expression can come handy for some fallback for a machine learning model. It has the power of pattern-matching, which can ensure that the data we are processing is correct or incorrect. Most of the early chatbots discussed in Chapter 1 under the History of Chatbots section were hugely dependent on pattern-matching.

Let's take below two examples that are pretty simple to understand. We'll try to use regular expression to extract information from both of the sentences.

Book me a metro from Airport Station to Hong Kong Station.

Book me a cab from Hong Kong Airport to AsiaWorld-Expo.

Here is the code:

```
sentence1 = "Book me a metro from Airport Station to Hong Kong Station."
sentence2 = "Book me a cab to Hong Kong Airport from AsiaWorld-Expo."
import re
from_to = re.compile('.* from (.*?) to (.*?)')
to_from = re.compile('.* to (.*?) from (.*?)')
from_to_match = from_to.match(sentence2)
to_from_match = to_from.match(sentence2)
if from_to_match and from_to_match.groups():
    _from = from_to_match.groups()[0]
    _to = from_to_match.groups()[1]
    print("from_to pattern matched correctly. Printing values\n")
    print("From: {}, To: {}".format(_from, _to))
elif to_from_match and to_from_match.groups():
    _to = to_from_match.groups()[0]
    _from = to_from_match.groups()[1]
    print("to_from pattern matched correctly. Printing values\n")
    print("From: {}, To: {}".format(_from, _to))
```

Output:

```
to_from pattern matched correctly. Printing values
From: AsiaWorld-Expo., To: Hong Kong Airport
```

Try changing the `sentence2` to `sentence1` and see if the code works well to identify the pattern. Given the power of machine learning these days, regular expression and pattern-matching has taken a back step, but make sure you brush up a bit about it as it may be needed at any time to parse specific details from words, sentences, or text documents.

Summary

At this point you must have fair idea of why we need to know NLP before starting to build chatbots. In this chapter, we learned about the spaCy module in Python, its features, and how to install it. We dived into various methods of NLP, which is extensively used while building chatbots. We learned about POS tagging, the difference between stemming and lemmatization, entity recognition, noun-chunking, and finding similarity between sets of words.

We executed code for all of these concepts and learned all of this by doing and not just reading, which is what this book emphasizes. We brushed up on basics of tokenization and regular expressions as well. We are good to go on and build our chatbot in the next chapter using a freely available tool called Dialogflow. We will learn how to train our chatbot to understand and extract information given by the user in the next chapter.

Footnotes

¹ <https://catalog.ldc.upenn.edu/ldc2013t19>
